Graylinx

Industrial Platform

Master Product & Architecture Specification

v1.1.0 — Draft (Phase 1)

May 2026

Document Scope
PRD — Product vision, problem, goals, personas, success metrics
Architecture — System design, data flow, components, deployment
Data Model — Database schema, Data Access Model, time-series storage
Core Modules — Monitoring, Alerting, Reporting, Work Orders, Scheduling, Portfolio Mgmt, Integration
Digital Twin — Commissioning baseline approach, lifecycle management, FDD integration
Reinforcement Learning — Optimization integration, safety bounds, twin coupling
Agentic AI — Multi-agent system (Planner / Executor / Validator), tool registry, approval tiers
Data Quality — Null-handling contract, alert pre-checks, ETL quality jobs (right-sized for this architecture)
LLM Integration — Provider strategy, cost model, prompt patterns
HVAC Extension — Domain-specific modules for HVAC operations
Factory Extension — Monitoring, OEE, and maintenance for both discrete and process manufacturing
Roadmap — 24-month phased delivery plan
Glossary, design decisions log, open questions

Executive Summary

Graylinx Industrial Platform is a domain-agnostic, on-premise IoT operations platform for industrial sites. It unifies real-time monitoring, predictive maintenance via digital twins, optimization via reinforcement learning, autonomous workflow execution via Agentic AI, and operations workflows in one system. It deploys across 50–500 sites per customer, with 100–1,000+ devices per site.

What Makes Graylinx Different

- Domain-agnostic core with vertical extensions — single platform serves HVAC, factory floor, water, power distribution, and other industrial verticals.
- Digital Twin Fault Detection based on commissioning baseline — twin trained on first 2 weeks post-commissioning, used as lifetime reference. Versioned baselines preserved across the equipment lifetime.
- Independent FDD application reads ETL'd data, looks at multi-point error patterns over time. Naturally noise-resistant by design.
- Reinforcement Learning agents optimize equipment continuously within safety bounds. Mostly binary state observations + real-time building load.
- Agentic AI framework — multi-agent system (Planner / Executor / Validator) handles routine workflows autonomously with explicit approval gates for high-risk actions.
- Data quality is absorbed structurally by design (5-min poll + CoV, null-instead-of-zero, persistence in alerts, multi-point FDD), not by a heavy real-time quality service.
- Protocol-agnostic edge — BACnet, Modbus, OPC-UA, MQTT, REST, Profinet via pluggable adapters.
- On-premise deployment — data stays on customer infrastructure. No cloud lock-in.

Headline Numbers (Design Targets)

Target	Value
Sites per deployment	50–500
Devices per site	100–1,000+
Data acquisition cadence	5-minute poll + CoV (Change of Value) for fast-changing signals
Real-time UI latency	< 5 seconds (telemetry to dashboard)
API response (recent data)	< 400 ms for last 30 days
Analytics query (historical)	< 2 seconds for >30 day data from DWH
Alert delivery	< 30 seconds from threshold breach (with persistence check)
Twin baseline window	2 weeks of post-commissioning data

Twin retrain trigger	Step-change in error toward baseline (configurable: auto or semi-auto approval)
RL optimization improvement	10–25% on tracked KPI within 30 days of deployment
Agentic AI autonomy	60%+ of routine workflows completed without human intervention
Pipeline uptime	≥ 95% per site

Document Map

This document is organized into 13 parts. Each part is self-contained but cross-references related sections.

Part	Topic	What's Inside
1	Product Requirements (PRD)	Vision, problem statement, goals, success metrics, user personas, scope boundaries
2	System Architecture	Layered architecture, data flow end-to-end, technology stack, deployment topology
3	Data Access Model & Storage	DAM concepts, database schema, time-series storage, multi-tenancy
4	Core Functional Modules	Monitoring, Alerting, Reporting, Work Orders, Scheduling, Portfolio, Integration, Commissioning Tool
5	Digital Twin & FDD	Commissioning baseline, lifecycle states, FDD app design, baseline versioning
6	Reinforcement Learning	RL broker, safety bounds, deployment modes, twin coupling
7	Agentic AI Framework	Multi-agent system, tool registry, approval tiers, standard workflows
8	Data Quality Strategy	Why ETL is sufficient + null contract + alert pre-checks + ETL quality jobs
9	LLM Integration	Provider strategy, cost model, prompt patterns, validation, privacy
10	HVAC Extension	HVAC-specific modules, fault codes, energy optimization, BMS integration
11	Factory Extension (slim)	Production monitoring, OEE, maintenance — discrete + process manufacturing
12	Roadmap	Phase 0 foundation through Phase 3 v3 — 24-month delivery plan, success metrics, risks
13	Appendix	Glossary, design decisions log, open questions

PART 1 — Product Requirements

1.1 Vision

Graylinx Industrial Platform unifies the operational stack of distributed industrial sites in one on-premise platform. It eliminates data silos, replaces reactive maintenance with predictive twin-based fault detection, replaces manual operations with autonomous AI workflows, and continuously optimizes equipment performance via reinforcement learning.

The product is domain-agnostic at its core, with extensions for specific verticals (HVAC, factory floor, water treatment, power distribution). One platform, many industries, one source of truth per customer.

1.2 Problem Statement

Operations teams managing distributed industrial sites face six critical challenges:

- Fragmented Systems — Multiple vendor platforms (BMS, SCADA, CMMS, ERP, MES) don't communicate. No unified view across the portfolio.
- Reactive Maintenance — Equipment failures detected after breakdown. Threshold alerts catch symptoms after damage is done. Mean Time to Detect (MTTD) measured in hours or days.
- Static Operations — Setpoints, schedules, and control parameters are fixed once and rarely revisited. Operations drift away from optimal as conditions change.
- Manual Workflows — Even with alerts, humans must investigate, decide, and execute every response. Operations teams spend most of their time on routine, repeatable tasks.
- Compliance Risk — Audit trails, performance metrics, incident logs scattered across systems. Hard to prove what happened and why.
- Cost Overruns — Energy waste, unplanned downtime, inefficient technician dispatch, missed performance contracts.

1.3 Why Graylinx Solves This

Problem	Graylinx Solution
Fragmented systems	Single platform across all sites and equipment types. Integration adapters for existing BMS, SCADA, CMMS, MES — they become data sources, not silos.
Reactive maintenance	Digital Twin FDD detects deviation from commissioning baseline at root cause level — catches developing failures days/weeks before symptoms reach threshold-alert zone.
Static operations	Reinforcement Learning agents continuously tune setpoints, schedules, and control parameters within configured safety bounds. Equipment improves over time.

Manual workflows	Agentic AI orchestrates multi-step responses autonomously: detect → diagnose → plan → execute → validate. Approval gates only for high-risk actions.
Compliance risk	Immutable audit trails for every action — human or agent. Versioned baselines preserve equipment history. Quality flags on every reading.
Cost overruns	RL optimization reduces energy 10–25%. Predictive maintenance reduces unplanned downtime 30%+. Agentic AI reduces operations workload 60%+.

1.4 Goals & Success Metrics

Business Goals

- Single system of record for operations across the customer's entire portfolio
- Reduce Mean Time to Detect (MTTD) from hours to under 30 minutes via twin FDD
- Reduce Mean Time to Repair (MTTR) by 30%+ via Agentic AI auto-diagnosis and contextual data
- Enable 15–25% operational cost reduction via RL-driven optimization
- Reduce manual operations workload by 70% via Agentic AI autonomous workflows
- Support 50–500 sites with sub-second telemetry response and consistent UX

MVP Success Metrics (Month 6)

Metric	Target
Telemetry pipeline uptime per site	≥ 95%
Portfolio dashboard load time (500 sites)	< 2 seconds
Real-time UI latency	< 5 seconds
Alert delivery (rule-based + twin)	< 30 seconds from anomaly
Twin FDD coverage	Detect 80%+ of equipment faults ≥24h before failure
RL optimization	10%+ improvement on tracked KPI within 30 days
Agentic AI autonomy	60%+ routine workflows autonomous, < 5% incorrect actions
Work order creation-to-dispatch	< 10 minutes
Beta customer sign-off	After UAT in Month 6

1.5 User Personas

Graylinx serves five primary personas plus domain specialists. Each has distinct workflows, devices, and information needs.

Portfolio Manager (Primary)

Oversees 50–500 sites. Reviews exception alerts, approves Agentic AI high-risk actions, monitors SLA compliance and cost trends. Works on desktop. Checks dashboards 3–5x/day. Cares most about uptime, SLA compliance, cost reduction, team productivity.

Site Operator (Primary)

On-site daily. Operates tablet or kiosk to monitor device status and act on alerts. Validates twin diagnoses, accepts or overrides Agentic AI recommendations. Creates and closes work orders, escalates to senior technicians.

Operations Engineer (Secondary)

Analyzes operational data, tunes digital twin models, configures RL reward functions, audits Agentic AI decisions. Needs detailed historical data, model performance metrics, and override capabilities. Desktop-first.

Maintenance Technician (Secondary)

Receives dispatched work orders (often auto-generated by Agentic AI from twin diagnoses), executes repairs, logs outcomes. Primary device: tablet in field. Receives twin diagnosis context, recommended fixes, parts lists, AI-generated guidance.

AI Operations Specialist (NEW Role)

This is a new role created by Graylinx. Configures and tunes Agentic AI agents, RL reward functions, and digital twin baselines. Reviews agent decision logs, validates retrain decisions in semi-automatic mode, manages baseline versioning. Bridges domain expertise and AI capabilities. Essential for v2/v3 maturity.

Domain-Specific Specialists (Tertiary)

- HVAC: Energy Manager — monitors consumption, drives RL-led optimization, demand response.
- HVAC: Senior Technician — handles complex faults, supervises junior staff.
- Factory: Production Manager — monitors OEE, throughput, identifies bottlenecks.
- Factory: Maintenance Lead — manages PM schedules, prioritizes repairs.

1.6 Out of Scope

To prevent scope creep, the following are explicitly NOT in Graylinx scope:

- Replacing customer's ERP, CRM, or financial systems
- Replacing customer's MES (Graylinx integrates with MES, doesn't replace it)
- Replacing customer's quality management system in factories
- Replacing safety interlock or emergency-shutdown systems (these stay on dedicated SIS)
- Building digital twins from scratch — Graylinx integrates customer-built twin modules
- Building RL agents from scratch — Graylinx integrates customer-built RL agents
- Cloud-only deployment — on-premise is the design center; cloud is a deployment option, not the default
- End-user financial reporting (Graylinx feeds BI tools but isn't a BI replacement)

PART 2 — System Architecture

2.1 Architectural Principles

- Layered separation — protocol, transformation, streaming, storage, intelligence, application, presentation are independent layers.
- Event-driven — Kafka streams telemetry between producers and consumers; consumers operate independently with their own offsets.
- Domain-agnostic core, vertical extensions — HVAC and factory are extensions, not separate products.
- On-premise first — designed to run inside customer's network perimeter; cloud deployment is an option, not assumed.
- Pluggable everywhere — protocol adapters, integration connectors, agent tools, twin models, RL agents are all pluggable.
- Existing-system integration over replacement — Graylinx integrates with BMS, CMMS, MES, ERP rather than replacing them.
- Quality absorbed structurally — 5-min poll + CoV, null-instead-of-zero, multi-point FDD, persistence-based alerting do most of the quality work without a heavy real-time quality service.

2.2 Architecture Layers (Logical)

Layer	Purpose	Components
L1 — Edge / Source	Connect to physical equipment and external systems	DDCs, PLCs, Protocol Converters, REST APIs, Webhooks
L2 — Protocol	Speak each protocol natively, normalize to internal schema	BACnet, Modbus, OPC-UA, MQTT, REST, Profinet adapters
L3 — Transform	Unit conversion, scaling, business mapping per Data Access Model	Transformer Module, CoV check, DAM lookup
L4 — Stream	Distribute raw + transformed data to consumers	Kafka (multi-instance, partitioned), Redis (jobs, cache)
L5 — Storage	Persist config + time-series + analytics	PostgreSQL (transaction), TimescaleDB (hypertable), Data Warehouse, Data Lake
L6 — Intelligence	Twin FDD, RL agents, Alerting, Agentic AI workflows	Twin Broker, RL Broker, Alert Engine, Agentic AI Orchestrator, FDD App
L7 — Application	Business APIs, scheduling, configuration	REST + GraphQL API, Scheduler, Commissioning Tool, ETL Service
L8 — Presentation	Web UI, real-time updates, mobile/tablet	React SPA, WebSocket, Tablet/Kiosk UI

L9 — Auth (cross-cutting)	Identity, RBAC, multi-tenancy	Keycloak, JWT, RBAC engine
L10 — Observability (cross-cutting)	Metrics, logs, traces	Prometheus, Grafana, structured logging

2.3 End-to-End Data Flow

From a device reading to a UI update, including all integration points:

Data Flow (Step-by-Step)
1. Device emits reading via BACnet / Modbus / Profinet / OPC-UA / REST / Webhook
2. Protocol Adapter parses and normalizes to Graylinx point schema
3. CoV Check filters unchanged values (heartbeat every 5 min guarantees liveness)
4. Transformer Module applies unit conversion, scaling, derived metrics (uses DAM from Transaction DB)
5. When device offline → null delivered (NEVER coerced to 0); MISSING flag attached
6. Kafka receives event (raw + transformed + metadata)
7. Kafka fans out to multiple consumer groups in parallel:
7a. TimescaleDB Writer → hypertable storage
7b. WebSocket Service → live UI clients
7c. Twin Broker → digital twin (in NORMAL state) for prediction
7d. RL Broker → state observation for RL agents
7e. Alert Engine → real-time threshold + persistence rules
7f. Agentic AI Trigger → workflows depending on real-time data
8. Scheduler runs ETL jobs on schedule (15 min / hourly / daily) — populates DWH from raw + transformed
9. ETL also runs quality jobs (drift, cross-sensor, baseline scoring) — feeds back to twin calibration
10. FDD App reads ETL'd data, looks at multi-point error patterns over time, classifies faults
11. UI consumes 3 channels: WebSocket (live) + API (last 30 days from hypertable) + Analytics endpoint (>30 days from DWH)
12. Commissioning Tool writes configs to Transaction DB → broadcasts on config.changes Kafka topic → Protocol Layer + Transformer pick up new configs

2.4 Technology Stack

Layer	Technology	Rationale
Frontend SPA	React 18, TypeScript, Vite, Tailwind	Fast, typed, modular, mature ecosystem
Backend API	Node.js, Fastify, TypeScript	Lightweight, fast, JSON-native, same language as frontend
API style	REST + GraphQL	REST for integrations, GraphQL for flexible UI queries
Real-time	WebSocket (Fastify-WS)	Push live telemetry to clients with low latency
Protocol adapters	node-bacstack, modbus-serial, mqtt.js, node-opcua, snap7 (Profinet)	Multi-protocol coverage, mature libraries
Streaming	Apache Kafka (multi-instance)	Durable, partitioned fan-out for telemetry; replay capability
Cache / Jobs	Redis + BullMQ	Job queues, caching, ephemeral data — Kafka is overkill for these
Transactional DB	PostgreSQL 15+	Configuration, users, work orders, audit, DAM
Time-series DB	TimescaleDB (PostgreSQL extension)	Hypertables, compression, time-based partitioning, SQL-compatible
Data Warehouse	PostgreSQL DW or ClickHouse (Phase 2)	Long-range analytics, ETL target
Data Lake	S3-compatible (MinIO on-prem) — Phase 2	Raw archival, ML feature store
Auth	Keycloak (self-hosted)	SSO / SAML / OIDC, AD/LDAP federation, multi-tenant realms
Twin runtime	Customer-built (Python, PyTorch / TF)	Customer-owned models, integrated via Twin Broker
RL runtime	Customer-built (Python, RLlib / Stable-Baselines)	Customer-owned agents, integrated via RL Broker
FDD app	Python, FastAPI, scikit-learn	Multi-point error pattern detection on ETL'd data
LLM (Agentic AI backbone)	Anthropic Claude (Sonnet / Opus)	Long context, tool use, reasoning, on-prem option (v3)
Container runtime	Docker	Standard, portable, well-tooled
Orchestration	Docker Compose (small) / Kubernetes (production)	Compose for ≤50 sites, K8s for 50–500
Observability	Prometheus + Grafana, Loki / Elasticsearch	On-prem metrics, logs, dashboards

2.5 Kafka Topic Design

Topic	Partitioning	Consumers
telemetry.raw	By site_id	TimescaleDB writer, audit
telemetry.transformed	By site_id + equipment_id	WebSocket, Twin Broker, RL Broker, Alert Engine, Agentic AI
alerts.fired	By severity	Notification gateway, Agentic AI, dashboards
commands.write	By site_id	Protocol Adapters (write-back from RL / Agent / human)
config.changes	Single partition	Protocol Layer, Transformer (config refresh on change)
twin.lifecycle	By equipment_id	FDD App, RL Broker, Alert Engine, dashboards (state changes)
agent.activity	By workflow_id	Agent activity feed UI, audit log

2.6 Deployment Topology

Single-Server (Dev / Small Org)

Docker Compose, all services on one server. Suitable for ≤ 50 sites, dev environments, demo deployments. Single Postgres, single Kafka, single Redis.

Kubernetes Cluster (Production)

Multi-node Kubernetes on customer's private cloud or on-prem datacenter. Suitable for 50–500 sites.

- Helm charts for repeatable deployment
- Horizontal scaling: API pods, adapter pods, ETL workers, twin broker, RL broker
- Postgres replication (primary + standby), Kafka multi-broker (3+), Redis sentinel
- Persistent volumes for time-series data with automated backup
- GPU node pool optional for twin physics simulation and RL inference

Edge + Central (Hybrid)

Protocol adapters at the edge per site. Central platform aggregates. For very large portfolios, poor WAN connectivity, or strict on-site data residency.

Component	Edge (per site)	Central (HQ)
Protocol adapters	Run locally	—
Local cache	24h buffer for WAN outage	—
Kafka	—	Central cluster
Storage	—	Central PostgreSQL + TimescaleDB
Twin / RL	—	Central (or per-site for low-latency control)
UI	Cached static assets	Served centrally

PART 3 — Data Access Model & Storage

3.1 The Data Access Model (DAM) Concept

The Data Access Model is the structured definition of which physical inputs map to which logical points, with all metadata required to interpret a reading correctly. It is the single source of truth that every other subsystem consults.

DAM is built and maintained via the Commissioning Tool. Once configured, it drives the runtime behavior of Protocol Layer, Transformer, FDD App, Twin Broker, RL Broker, Alert Engine, and UI.

3.2 DAM Concepts

Concept	Identifies	Examples	Storage
Source	Hardware connecting devices	DDC, PLC, Protocol Converter	sources table
Device	Physical equipment	Chiller-1, AHU-North, CNC-12	devices table
Param (Point)	Specific measurement on device	supply_temp, discharge_pressure, RPM	params table
Units	Engineering unit + conversion	°C, bar, RPM, kW	units table
Transformer Metric	Conversion formula raw → engineering	raw × 0.1 + offset	transformer_rules
Site	Geographic location	Plant-A, Building-B	sites table
Location / Building	Sub-site grouping	Tower-1, Wing-A	buildings table
Grouping	Functional grouping	Floor-3, Zone-B, Production-Line-2	groups table
Mapping	device + param + source binding	Chiller-1 / supply_temp / DDC-7 / Modbus reg 40001	device_param_source
Protocol	Communication protocol per source	Modbus TCP, BACnet/IP	protocol_config
Time-Series Table	Where readings are stored	telemetry hypertable	TimescaleDB

3.3 Database Schema — Configuration Tables

Sites & Hierarchy

Table	Key Columns
tenants	tenant_id (UUID PK), name, created_at, billing_plan, config_json
sites	site_id (PK), tenant_id (FK), name, lat, lon, address, timezone, created_at
buildings	building_id (PK), site_id (FK), name, type (office / plant / warehouse), floor_count
groups	group_id (PK), building_id (FK), name, type (floor / zone / line / area), parent_group_id (self-FK for nesting)

Devices & Points

Table	Key Columns
sources	source_id (PK), site_id (FK), type (DDC / PLC / Protocol_Converter), vendor, model, ip_address, protocol
devices	device_id (PK), group_id (FK), name, equipment_type, manufacturer, model, serial_number, commissioning_date, twin_model_id (nullable)
params	param_id (PK), name, description, default_unit, point_type (temperature / pressure / flow / power / status), is_writable
units	unit_id (PK), name, symbol, conversion_to_si, base_unit_id (nullable)
device_param_source (mapping)	mapping_id (PK), device_id, param_id, source_id, register_or_object, scale_factor, offset, poll_interval_seconds, cov_threshold
transformer_rules	rule_id (PK), mapping_id (FK), formula, output_unit_id, calibration_offset, drift_coefficient, last_calibrated_at
protocol_config	config_id (PK), source_id (FK), protocol_type, port, auth_method, polling_strategy, retry_policy

Twin & Lifecycle

Table	Key Columns
twin_models	model_id (PK), equipment_type, model_artifact_path, version, framework (PyTorch / TF), input_schema, output_schema
twin_baselines	baseline_id (PK), device_id, model_id, version, training_start, training_end, captured_after_repair_wo_id (nullable), quality_score, equipment_age_at_capture, validation_metrics (JSONB), archived_at (nullable, NULL = active)
twin_lifecycle_state	device_id (PK), current_state (COMMISSIONING / TRAINING / NORMAL / DEGRADING / FAULT / RECOVERING / RETRAIN_PENDING / RETRAINING / RETIRED), state_entered_at, current_baseline_id (FK), retrain_mode (auto / semi_auto), updated_at
twin_lifecycle_transitions	transition_id (PK), device_id, from_state, to_state, transition_time, triggered_by (system / user_id / agent_id), reason, related_wo_id (nullable)
twin_error_trend (hypertable)	time, device_id, baseline_id, mean_error, max_error, error_classification (gradual / step / oscillating), sample_count, good_sample_count
fault_classifications	classification_id (PK), device_id, classified_at, fault_type, error_pattern, confidence, related_wo_id (nullable), source_app (FDD / agentic / manual)
step_change_events	event_id (PK), device_id, detected_at, error_before, error_after, persistence_days, trigger_action (auto_retrain / pending_approval), approved_by (nullable), approved_at (nullable)

Alerts, Work Orders, RL

Table	Key Columns
alert_rules	rule_id (PK), name, scope (device / group / site), condition (JSONB: threshold / persistence / etc), severity, escalation_chain, enabled
alerts (hypertable)	time, alert_id, rule_id, device_id, severity, status (open / ack / resolved / closed), value_at_breach, message, acknowledged_by, resolved_by, resolved_at
work_orders	wo_id (PK), tenant_id, site_id, device_id (nullable), title, description, priority, status (open / assigned / in_progress / resolved / closed), assigned_to (user_id), created_by (user_id or agent_id), created_at, due_at, closed_at, source (manual / alert / agent / pm / twin_diagnosis)
rl_agents	agent_id (PK), name, scope (device / group / site), kpi_target, reward_function (JSONB), policy_artifact_path, mode (shadow / production / paused), safety_bounds (JSONB)
rl_actions (hypertable)	time, agent_id, device_id, observation_vector, action_taken, reward_received, policy_version, was_executed

Agentic AI

Table	Key Columns
agent_workflows	workflow_id (PK), name, trigger_type (alert / schedule / manual / event), workflow_template (JSONB), enabled
agent_runs	run_id (PK), workflow_id, started_at, ended_at, status (running / completed / failed / awaiting_approval), trigger_context (JSONB), final_output (JSONB)
agent_steps	step_id (PK), run_id (FK), step_number, agent_role (planner / executor / validator), tool_called, tool_input, tool_output, llm_input_tokens, llm_output_tokens, started_at, ended_at
agent_approvals	approval_id (PK), run_id (FK), step_id (FK), action_proposed, tier (1-5), requested_at, approved_by (user_id, nullable), approved_at (nullable), decision (approved / rejected / timeout)
agent_tools	tool_id (PK), name, description, schema (JSONB), implementation_endpoint, max_tier (highest tier this tool can serve), enabled

Data Quality (right-sized for this architecture)

Table	Key Columns
sensor_health_scores	device_param_id, score (0-100), drift_pct, availability_pct, frozen_event_count, last_calibration_at, computed_at — populated by ETL drift estimation job
sensor_drift_coefficients	device_param_id, drift_coefficient, bias_offset, last_estimated_at — pushed back into transformer_rules at runtime
device_offline_events	event_id, device_id, offline_at, online_at, duration_seconds, root_cause (network / device / adapter)
frozen_sensor_events	event_id, device_param_id, started_at, ended_at, frozen_at_value, sample_count, source_app (alert_engine / rl_broker)

3.4 Time-Series Storage (TimescaleDB)

telemetry hypertable

Column	Description
time	TIMESTAMPTZ — partition key, indexed
device_param_id	FK to device_param_source — identifies which point this reading is for
raw_value	DECIMAL nullable — original reading from device (NULL if device offline)
transformed_value	DECIMAL nullable — after transformer rules applied (NULL if device offline)
quality_flag	VARCHAR — GOOD / MISSING / SUSPECT (used minimally — primary use is MISSING for offline)
source_timestamp	TIMESTAMPTZ — timestamp from device (may differ from arrival time)
ingested_at	TIMESTAMPTZ — when Graylinx received it

NULL handling: when device is offline, raw_value and transformed_value are stored as NULL with quality_flag = 'MISSING'. NEVER coerced to 0. This is a system-wide contract enforced from ingestion through every consumer.

Hypertable Configuration

- Chunk interval: 1 day (typical for 5-min poll cadence)
- Compression: enabled after 30 days
- Retention: 2 years on hypertable, then archived to data lake
- Indexes: (device_param_id, time DESC), (time DESC) for range queries
- Continuous aggregates: hourly and daily rollups for fast analytics queries

3.5 Data Volume Estimates

Scenario	Volume
Sites	500 sites per deployment
Devices per site	500 (avg)
Points per device	10 (avg)
Total points	500 × 500 × 10 = 2.5M points
Polling cadence	Every 5 min (12 readings/hour)
Raw rate (without CoV)	2.5M × 12 = 30M readings / hour
After CoV filtering	~3-5M readings / hour (10x reduction typical)
Storage / day (after compression)	~5-10 GB/day for 500-site deployment

Storage / year (after compression)	~2-4 TB/year

3.6 Multi-Tenancy

Graylinx is multi-tenant by design. One deployment serves multiple customers (or multiple business units within an enterprise) with full data isolation.

- All tables include tenant_id, indexed and joined on every query
- Row-level security in PostgreSQL enforces tenant isolation at DB level
- Keycloak realms map 1:1 with tenants (or 1:N for enterprise sub-tenants)
- JWT carries tenant_id; API enforces it on every endpoint
- Kafka topics are shared but partitioned by tenant_id for compute isolation
- Twin and RL artifacts are tenant-scoped (model files in tenant-specific paths)

PART 4 — Core Functional Modules

Graylinx Core consists of 8 domain-agnostic modules. Each module is extended by HVAC and Factory verticals (Parts 10 and 11). Each module is independently testable and has clear ownership.

Module 1 — Real-Time Monitoring & Control

Feature	Description
Portfolio Dashboard	Multi-site health overview with roll-up KPIs. Status heatmap, alert count, equipment online/offline, twin lifecycle distribution, RL agent status, agent activity feed.
Site Dashboard	List all devices at a site with live status, latest readings, active alerts, twin state indicator, RL agent status per equipment.
Live Telemetry Streaming	WebSocket real-time push (5-min cadence + CoV updates between). Display live values, quality flag, timestamp, twin prediction overlay.
Device Detail View	Drill-down: live readings, 24h/7d/30d trends, twin overlay (predicted vs actual), active alerts, linked work orders, lifecycle history, baseline timeline.
Device Registry	Catalog all connected devices with metadata: model, serial, protocol, network address, parameters, twin model link, RL agent link, commissioning date.
Parameter Write-Back	Remote parameter changes (setpoint, mode, on/off). Required for RL agent actions and Agentic AI. Audit trail for all writes.
Historical Trending	Multi-timeframe charts (1h, 1d, 7d, 30d, custom). Min/max/avg/percentile statistics. Twin prediction band overlay.
Multi-Device Compare	Side-by-side comparison of similar equipment across sites or time periods.

Module 2 — Alerting & Predictive Maintenance

Combines real-time rule-based alerting with Digital Twin Fault Detection (covered in detail in Part 5). Real-time alert engine consumes telemetry directly from Kafka with persistence checks; FDD application runs on ETL'd data with multi-point pattern detection.

Feature	Description
Real-Time Alert Engine	Reads from telemetry.transformed Kafka topic. Evaluates threshold + persistence rules. Pre-checks: hard physical limits (sensor fault detection), frozen sensor detection, null awareness.
Persistence-Based Rules	Threshold breach must persist for N consecutive readings before firing alert. Eliminates spike-triggered false positives without needing a quality layer.
Twin FDD Integration	Independent FDD app reads ETL'd data. Multi-point error pattern detection. Classifies fault type and feeds into alert engine + Agentic AI workflows. Detail in Part 5.
Hard Physical Limits Pre-Check	Before evaluating equipment alert rules, check value is within physical possibility (e.g., -50 to 100°C for typical HVAC sensors). Outside limits → SENSOR_FAULT alert (different category, different routing).
Frozen Sensor Detection	5–10 sample window of identical values → SENSOR_FROZEN alert routed to calibration team. Suppresses normal threshold alerts on frozen channel.
Alert Inbox & Management	Unified queue of rule-based + twin FDD + sensor alerts. Filter by severity, fault type, equipment, status. Acknowledge, resolve, close, link to work order.
Alert Escalation	Unacknowledged alerts escalate to higher tier after configurable timeout. Critical twin diagnoses escalate immediately. Configurable escalation chain per rule.
Fault Code Library	Mapped library of fault codes → human-readable descriptions, twin-identified root causes, recommended actions, parts lists.
Alert Grouping	When equipment in twin FAULT state, threshold alerts for same equipment grouped as duplicates of known fault. Prevents alert fatigue.
Notifications	Email, SMS, in-app push, webhook. Configurable channels per user, role, alert type, severity. Silenced for routine alerts during agent workflow execution.
Audit Trail	Every alert + acknowledgement + resolution logged with timestamp, user/agent, before-after state. Immutable.

Module 3 — Operations Analytics & Reporting

Feature	Description
Agent-Generated Reports	Agentic AI (Part 7) generates executive summary, health, performance reports. Validator agent ensures factual accuracy by cross-checking against DB.
Twin Diagnostic Reports	Per-equipment twin analysis: health score, lifecycle state, baseline age, error trend, predicted RUL, recommended actions, lifetime degradation vs commissioning baseline.
RL Performance Reports	Agent-by-agent: reward improvement, KPI gains, action distribution, learning curves, comparison vs baseline operation.
Custom Reports & Export	User-selectable metrics, time range, format (PDF, CSV, HTML, Excel). Scheduled or on-demand. LLM-assisted report design.
Comparative Analysis	Compare sites/devices/time periods. Twin and RL performance benchmarking. Equipment lifetime degradation tracking.
Data Visualization	Line charts, bar charts, heatmaps, twin overlays, RL reward curves, agent activity feeds, lifecycle Sankey diagrams.
Performance Analytics	Uptime %, SLA compliance, incident response time, technician productivity, cost metrics, energy KPIs (HVAC), OEE (factory).
Compliance Reports	Audit-ready exports of all actions (human + agent). Regulatory templates for industry-specific compliance.

Module 4 — Work Order & Task Management

Feature	Description
Work Order Lifecycle	States: open → assigned → in_progress → resolved → closed. Each transition logged with user, timestamp, notes.
Twin-Triggered WOs	Twin FDD diagnosis auto-creates WO with: fault type, root cause, RUL estimate, recommended parts, technician skill match, linked baseline, similar past faults.
Agent-Generated WOs	Agentic AI creates WOs during alert investigation workflows. Pre-populated with diagnosis and recommended actions. Validator confirms before creation.
Smart Technician Assignment	Agentic AI suggests best technician based on skills, current load, location, equipment familiarity, past success rate on similar issues.
Mobile Work Order UI	Tablet-optimized form. Technician sees twin diagnosis, recommended fix, parts list, manual references. Logs notes, photos, status updates from field.
Work Order Analytics	Completion time, SLA compliance, technician performance, repeat issue tracking, twin diagnosis accuracy.
Preventive Maintenance Scheduling	Auto-generate recurring WOs from PM templates. Twin RUL estimates inform PM timing (PM when twin says RUL is approaching threshold, not on fixed calendar).
CMMS Integration	Bi-directional sync with Maximo, ServiceNow, UpKeep, IBM Maximo. Single source of truth — Graylinx is master, CMMS is mirror, or vice versa (configurable per customer).
Parts Inventory Link	Twin diagnosis suggests parts needed. WO links to inventory system to verify availability before dispatch.

Module 5 — Intelligent Scheduling & Automation

Feature	Description
Schedule Builder UI	Visual tool to define on/off schedules by time, day, week, month. Apply to device groups. Holiday calendars, exception schedules.
Automated Execution	Scheduler service applies schedule changes at designated times via parameter write-back through Kafka commands.write topic.
Agent-Driven Scheduling	User goal in natural language: 'Schedule maintenance Q3, avoid occupancy hours, balance tech load.' Agentic AI generates optimal schedule.
Conflict Detection	Detects scheduling conflicts (resource, technician, equipment) and proposes alternatives.
Workload Balancing	Distribute tasks across technicians fairly. Optimize for travel time, skill match, current workload.
RL-Optimized Schedules (v3)	RL agents learn optimal schedules: when to run/pause equipment, sequence of operations, batch sizing.
Schedule Templates	Pre-defined common schedules. Clone and customize per site. Versioned.

Module 6 — Portfolio & Multi-Site Management

Feature	Description
Site Hierarchy	Portfolio → Site → Building → Group → Device. Flexible nesting. Metadata per level.
User Management & RBAC	Roles: Admin, Portfolio Manager, Operator, Technician, Engineer, AI Ops Specialist, Read-Only. Granular permissions per resource type.
Agent Authorization Model	Define what agents can do autonomously vs require approval. Per-tier action permissions. Agent service tokens with scoped permissions.
Multi-Tenancy	Complete data isolation per customer. One platform serves many enterprises. Tenant-scoped data, configs, models, agents.
Configuration Templates	Reusable alert rules, schedules, twin models, RL agents, agent workflows. Apply to new sites with one click.
Audit Logging	Immutable log of all user + agent actions. Twin lifecycle changes, RL decisions, agent workflows fully logged. Compliance-ready exports.
Data Governance	Retention policies, encryption at rest and in transit, RBAC at API and DB layer, data residency controls.

Module 7 — Integration & Extensions

Feature	Description
Protocol Adapter Framework	Pluggable adapters for BACnet, Modbus, OPC-UA, MQTT, REST, Profinet. Standardized interface; new protocols added without core changes.
Domain Extension Framework	HVAC Extension, Factory Extension, etc. Extensions add domain-specific data models, alert rules, reports without core bloat.
Twin Module Integration	Twin-broker service connects to customer-built twin modules. Standardized twin data schema. Multi-twin support per device.
RL Agent Integration	Agent-broker service hosts customer-built RL agents. Standardized state/action/reward interface.
Agentic AI Tool Registry	Register new tools for agents to call. Schema-defined inputs/outputs. Versioned and authorized per approval tier.
External System Integration	BMS bridge (Siemens / Honeywell / JCI), MES connector, CMMS sync (Maximo / ServiceNow), ERP integration (SAP / Oracle). Webhook + polling.
REST API & SDK	Full OpenAPI 3.0 spec. JavaScript and Python SDKs for custom integrations.
GraphQL API	Flexible queries for UI and analytics. Reduces over-fetching. Subscription support for real-time.
Data Export	Export to Tableau, Power BI, data warehouse, S3. CSV, JSON, Parquet formats. Scheduled or on-demand.
Notification Gateway	Email, SMS, in-app, webhooks, Microsoft Teams, Slack. Extensible to custom channels.

Module 8 — Commissioning Tool

First-class component for onboarding new sites and devices. Maintains the Data Access Model. Drives runtime configuration of every other component.

Feature	Description
Source Configuration	Add DDC / PLC / Protocol Converter. Specify protocol, network address, authentication. Test connectivity before save.
Device Registration	Add devices to a site. Specify equipment type, manufacturer, model, serial, commissioning date.
Point Mapping	Map physical inputs (Modbus registers, BACnet objects) to logical points (params). Set unit, scale factor, offset, poll interval, CoV threshold.
Bulk Import	CSV / Excel import for large sites. Template-based for common equipment types.
Validation & Test	Test polling for each new mapping. Show live values during commissioning. Validate transformer rules produce sensible engineering units.
Twin Model Assignment	Assign twin model to device. Sets up twin lifecycle in COMMISSIONING state. Triggers 2-week baseline window.
RL Agent Assignment	Assign RL agent to device or group. Configure reward function, safety bounds, mode (shadow / production).
Site / Building / Group Hierarchy	Define geographic and functional hierarchy. Group devices by floor, zone, line, area.
Live Config Push	Save broadcasts on Kafka config.changes topic. Protocol Layer, Transformer, Alert Engine pick up changes within seconds.
Versioning & Rollback	All config changes versioned. Rollback to previous config if new mapping causes issues.

PART 5 — Digital Twin & Fault Detection

5.1 Approach

Graylinx uses customer-built digital twin modules anchored to the equipment commissioning baseline. The approach is fundamentally different from continuous-prediction physics-based FDD systems:

- Twin is trained on first 2 weeks of post-commissioning operation — this is the lifetime reference for what 'normal' looks like for this specific equipment.
- Twin remains frozen on this baseline through normal operation. It is NOT continuously updated.
- An independent FDD application reads ETL'd data, compares actual operation to twin prediction, looks at error patterns across multiple points over time.
- When error pattern indicates fault, equipment enters DEGRADING then FAULT state. Twin remains frozen.
- After repair, FDD watches for step-change: error returning toward baseline indicates repair successful and equipment recovered.
- Step-change triggers retrain decision — automatic OR semi-automatic (engineer approves) per config.
- Retrain captures a new 1-week baseline. Old baseline archived (versioned, never deleted).
- Result: equipment has a versioned history of baselines from commissioning through every repair cycle. Lifetime degradation and ROI is measurable.

Why This Approach
Commissioning baseline captures 'as-installed' performance — the cleanest possible reference point
Twin frozen during normal operation means error trends are meaningful (not absorbed by continuous retraining)
FDD on ETL'd data with multi-point patterns is naturally noise-resistant — no real-time quality layer needed
Step-change detection handles the 'when to retrain' question deterministically
Versioned baselines enable equipment lifetime ROI analysis: 'how does this chiller compare to day 1?'
Twin frozen during fault period prevents contaminated retraining on degraded data

5.2 Twin Lifecycle States

State	Description
COMMISSIONING	Equipment newly installed. First 2 weeks of post-commissioning data being collected for baseline twin training. Twin not yet active. Quality monitored — extends window if quality < 95%.
TRAINING	Baseline window complete with sufficient quality. Twin model fitting on captured data. Validation against held-out portion of window. Independent FDD app inactive for this device.
NORMAL	Twin live. Predicts expected behavior given inputs. Independent FDD app monitors prediction error continuously on ETL'd data. Multi-point error patterns evaluated.
DEGRADING	FDD detects rising error pattern across multiple points. Equipment suspected of degrading. Twin frozen on current baseline (this is what enables fault detection — the gap between twin and reality IS the diagnostic signal).
FAULT	FDD has classified the error pattern as a confirmed fault. Twin held frozen. Repair work order created. RL agent paused or in conservative mode. Threshold alerts grouped to prevent fatigue.
RECOVERING	Repair WO closed. Equipment back in operation. FDD watches for step-change: error returning toward baseline-era levels with persistence (3+ days).
RETRAIN_PENDING	Step-change confirmed. Pending retrain decision: auto-trigger OR semi-auto (engineer approves) per config. UI surfaces pending approvals to AI Ops Specialist.
RETRAINING	New 1-week baseline window opens. Old baseline archived (versioned). Same quality requirements as COMMISSIONING. RL agent stays paused.
RETIRED	Equipment decommissioned. Twin and all baselines archived but inactive. Lifetime data preserved for fleet analytics.

5.3 State Transitions

Lifecycle State Transitions
COMMISSIONING (2 weeks data collection, ≥95% quality)
↓ baseline window complete + quality met
TRAINING (twin model fits + validates on captured data)
↓ training complete, validation passed
NORMAL ◄──┐
↓ FDD detects rising error pattern │
DEGRADING │
↓ FDD classifies pattern as confirmed fault │
FAULT (twin frozen, WO created, RL paused) │
↓ repair WO closed │
RECOVERING │
↓ step-change to near-baseline (3+ day persistence)
RETRAIN_PENDING │
↓ auto-trigger OR engineer approves │
RETRAINING (new 1-week baseline; old archived) │
└──┘
return to NORMAL with new baseline (versioned history preserved)

5.4 Independent FDD Application

The FDD application is architecturally separate from the twin runtime. Twin produces predictions; FDD app analyzes the error between predictions and actuals.

Why Separate?

- Twin and FDD have different update frequencies — twin runs continuously, FDD runs on ETL'd data (every 15 min / hourly)
- FDD logic evolves independently of twin models — better fault classification doesn't require retraining all twins
- FDD can correlate across multiple twins (cross-equipment patterns) — would be coupled and brittle if integrated
- FDD development team and twin development team can work independently

FDD Inputs

- ETL'd time-series data from data warehouse (cleaned, with drift correction applied)
- Twin predictions for the same time period
- Twin lifecycle state (only operates during NORMAL state for active classification)
- Sensor health scores (rules out sensor drift before classifying equipment fault)
- Equipment type and fault code library (for classification)

FDD Outputs

- Per-equipment error trend (mean error over time, multi-point error vector)
- Fault classifications (when error pattern matches a known fault signature)
- Lifecycle state transition recommendations (NORMAL → DEGRADING → FAULT)
- Step-change detection events (during RECOVERING state)
- Confidence score per classification

Multi-Point Error Pattern Analysis

FDD does not look at single-point deviations. It evaluates error vectors across all points of an equipment simultaneously, looking for known fault signatures:

Pattern	Indication
Single-point gradual rise	Likely sensor drift on that point — route to calibration team, NOT equipment fault.
Multi-point gradual rise (correlated)	Likely equipment degradation — fouling, wear, etc. Classify by which points and direction.
Step change in single point	Likely sensor failure or replacement event.

Step change across multiple points	Major equipment event (compressor failure, valve stuck, etc.).
Oscillating error	Control loop instability — RL or BMS controller issue.
Diverging error in pair (inlet/outlet)	Heat exchanger fouling, blocked filter, capacity loss.

5.5 Baseline Versioning

Every baseline is preserved forever. The current baseline is active; previous baselines are archived but accessible.

What's stored per baseline

- Baseline ID (UUID)
- Equipment ID
- Twin model artifact (file path, version, framework)
- Training data window (start, end)
- Quality score during training window (% GOOD readings)
- Equipment age at capture (days since commissioning)
- Triggered by (initial commissioning OR repair WO ID)
- Validation metrics (R^2, error std, multi-point coverage)
- Active flag (only one baseline active per equipment at a time)
- Archived timestamp (NULL if active)

Lifetime Comparison Capabilities

- Compare current twin error trend vs commissioning baseline → 'how degraded vs day-1?'
- Compare current twin error trend vs each post-repair baseline → 'how does each repair cycle perform?'
- Equipment ROI per repair: error reduction × repair cost = value
- Fleet analytics: 'which equipment models degrade fastest from commissioning?'
- Lifecycle insights: 'do we need to replace this asset or one more repair?'

5.6 Quality Requirements per Lifecycle State

State	Quality Requirement	If Quality Fails
COMMISSIONING	≥99% GOOD readings during baseline window. Any sustained quality issue requires window extension.	Extend window. Alert AI Ops Specialist. Do not transition to TRAINING.
TRAINING	Training data must be GOOD or high-confidence. SUSPECT and BAD excluded.	Return to COMMISSIONING and capture more data.
NORMAL	Twin inputs ≥97% GOOD. FDD considers quality flag when interpreting error.	Below threshold: alert AI Ops Specialist. Pause FDD if too degraded.
DEGRADING	Quality must rule out sensor drift before classifying equipment fault.	Sensor drift detected: route as SENSOR_DRIFT instead of equipment fault.
FAULT	Twin frozen, no quality dependency for FDD.	Quality issues during fault are independent — handled separately.
RECOVERING	Step-change detection requires ≥99% GOOD readings to avoid false recovery signal.	Spike or noise can falsely trigger step-change. Persistence (3+ days) required.
RETRAIN_PENDING	Engineer reviews quality scores during recent operation before approval.	Quality issues exist: defer retrain, fix sensors first.
RETRAINING	Same as COMMISSIONING — ≥99% GOOD over new 1-week baseline window.	Extend window if needed.

PART 6 — Reinforcement Learning

6.1 Approach

Graylinx integrates customer-built RL agents that continuously optimize equipment and plant operations. Agents learn from real operations within configured safety bounds and improve over time.

The RL Broker is the integration service that hosts agents, provides state observations from the telemetry stream, executes agent actions via the parameter write-back path, and tracks reward signals.

6.2 RL State Observations

RL agents in Graylinx work with two types of state observations:

Observation Type	Description
Binary State Signals	Most observations: equipment running/stopped, valve open/closed, occupied/unoccupied, mode (heating/cooling/auto), schedule active. Robust to most data quality issues by design.
Real-Time Building Load	Continuous value reflecting current load on the system. Used for context-aware action selection.

This design choice has important implications for data quality:

- Binary signals are inherently robust to spikes, drift, noise, range errors — these issues simply don't apply to 0/1 values
- The one risk for binary signals is frozen sensors (stuck at 1 or 0) — handled by a small frozen-detection check inside the RL Broker (~50 LOC)
- Real-time building load is more susceptible to gaps and spikes, but RL averages over time windows and equipment has natural inertia, so brief glitches rarely cause unsafe action
- Null handling: when a signal is null (device offline), RL Broker applies last-known-good or configured fallback policy — never propagates null to agent

6.3 RL Broker Responsibilities

Function	Description
Agent Hosting	Loads customer-built agents (Python, RLlib / Stable-Baselines / custom). Provides standardized state/action/reward interface.
Observation Assembly	Subscribes to telemetry.transformed Kafka topic. Assembles state vectors per agent's input schema.

Frozen Binary Detection	Inline check on binary signals: if signal hasn't changed in N samples (configurable, default 60 min for binary), flag as frozen and apply fallback.
Null Fallback	When telemetry is null (offline), use configured fallback: last-known-good (default) or safe-default (e.g., assume off).
Safety Bounds Enforcement	Hard limits on agent actions configured per-equipment. RL output clipped to bounds before write-back. E.g., never set chiller setpoint below 5°C.
Action Execution	Writes commands to commands.write Kafka topic. Adapter writes to device. Audit log records who/what/when.
Twin Lifecycle Awareness	Pauses agents when twin enters DEGRADING / FAULT / RECOVERING / RETRAINING. Resumes only after twin returns to NORMAL on new baseline.
Reward Tracking	Computes reward from KPI targets defined in agent config. Logs reward per action. Detects reward contamination (e.g., reward sensor BAD).
Mode Management	Shadow mode (recommendations only, not applied), production mode (actions applied), paused mode (no actions). Switching is logged.
Continuous Retraining	Agents periodically retrain on recent experience replay buffer. Replay buffer cleaned by ETL job (removes imputed/bad observations).
A/B Testing	Run RL agent vs baseline operation on subset of equipment. Statistical comparison.

6.4 Safety Bounds & Constraints

RL agents must operate within hard safety limits. Bounds are configured per equipment type and never crossed regardless of agent policy.

Bound Type	Example
Hard Range	Setpoint must be 16–28°C. Agent action clipped to this range before write-back.
Rate Limit	Setpoint cannot change more than 2°C per hour. Prevents agent from causing thermal shock.
Frequency Limit	Equipment cannot be cycled more than 4 starts per hour (compressor protection).
Cross-Equipment Constraints	If chiller-1 is at 100% load, chiller-2 must be running (capacity constraint).
Time-of-Day Constraints	Comfort setpoints during occupied hours; relaxed setpoints at night only.
Quality Bound	Quality (e.g., temperature deviation from target) must stay within tolerance. Reward heavily penalized for breaches.

6.5 RL Deployment Modes

Mode	Description
Shadow	Agent computes recommendations but actions are NOT applied. Used for: validating new agents, A/B comparison, building trust before promotion.
Human-in-the-Loop	Agent recommendations shown to operator who approves before execution. Used for: high-risk equipment, early production, regulated environments.
Production (Autonomous)	Agent actions applied automatically within safety bounds. Used for: validated agents on standard equipment.
Paused	Agent inactive. No observations processed, no actions taken. Used during: twin DEGRADING/FAULT/RECOVERING/RETRAINING, equipment maintenance, manual override.

6.6 Twin Lifecycle Integration

Twin State	RL Behavior
NORMAL	Full operation in production mode. Standard quality-aware observation handling.
DEGRADING	Reduce exploration. Avoid aggressive actions. Equipment is in unknown state — RL learning from this period would be contaminated.
FAULT	PAUSED. Hold conservative policy. Do NOT learn from fault-period observations.
RECOVERING	Stay paused. Do not resume until twin transitions to NORMAL on new baseline.
RETRAINING	Stay paused. Equipment characteristics may have changed — RL policy may need recalibration after twin retrain.
NORMAL (post-retrain)	Resume in shadow mode initially. Validate against new baseline before promoting back to production.

PART 7 — Agentic AI Framework

7.1 Approach

Graylinx replaces single-shot LLM calls with a multi-agent system where specialized agents collaborate on every significant operational task. The architecture is Planner / Executor / Validator.

Why Multi-Agent Instead of Single-Shot LLM
Separation of concerns: planning, doing, checking are distinct cognitive tasks; specialized agents perform each better than a single LLM
Reliability: Validator catches Executor errors before they cause damage. Hallucination mitigation built into architecture.
Auditability: Each agent's reasoning logged separately. Compliance and debugging are tractable.
Scalability: Agents can run in parallel for different tasks. Handoffs are explicit and traceable.
Safety: Validator gates high-risk actions. Human approval requested when uncertainty is high.
Cost control: Validator can short-circuit failed plans before expensive Executor steps.

7.2 Agent Roles

Planner Agent

Aspect	Detail
Input	Goal, alert, user request, or scheduled trigger
Output	Structured plan: steps, required tools, expected outcomes, approval gates, success criteria
Capabilities	Decomposition, dependency analysis, risk assessment, tool selection, approval-tier evaluation
LLM model	Claude Opus or Sonnet (heavy reasoning)
Token budget	Higher (4K–8K input, 1K–2K output for complex plans)

Executor Agent

Aspect	Detail
Input	Plan from Planner
Output	Tool call results, progress updates, completion report
Capabilities	Tool use (function calling), error handling, retries, progress tracking, intermediate result synthesis
LLM model	Claude Sonnet (fast, capable tool use)
Token budget	Variable per tool call; typically 2K–4K input + 500–1K output per call

Validator Agent

Aspect	Detail
Input	Executor results + original plan
Output	Validation verdict (success / partial / failed), discrepancy report, recommended next steps
Capabilities	Output verification, cross-checking against DB, anomaly detection, factual accuracy validation
LLM model	Claude Sonnet (independent perspective from Executor — different temperature)
Token budget	3K–5K input, 500 output (focused on verification)

7.3 Tool Registry

Tools are the interface between agents and the platform. Each tool has a defined schema, implementation endpoint, and approval-tier requirement.

Initial Tool Library

Category	Tools
Data Tools	get_telemetry, get_alerts, get_work_orders, get_equipment, get_twin_diagnosis, get_twin_lifecycle_state, get_rl_performance, get_baseline_history, get_quality_score
Control Tools	write_setpoint, change_mode, start_equipment, stop_equipment, override_schedule (all with approval gates)
Communication Tools	send_notification, send_email, create_work_order, assign_technician, request_approval
Reporting Tools	generate_report, export_data, create_dashboard_view, schedule_recurring_report
Analysis Tools	query_historical_trends, compare_periods, simulate_on_twin, predict_with_rl, classify_fault_pattern
Lifecycle Tools	transition_lifecycle_state, archive_baseline, trigger_retrain (with approval), pause_rl_agent, resume_rl_agent
Maintenance Tools	create_calibration_wo, schedule_pm, link_parts_to_wo, log_repair_outcome

7.4 Approval Tiers

Every action has an approval tier. Tiers determine whether human approval is required and how strict the validation.

Tier	Action Type	Approval	Examples
1	Read-only	None (autonomous)	Data queries, report generation, notifications
2	Low-risk write	None (autonomous)	Create work order, schedule task, log event, request approval
3	Operational write	Configurable (auto OR operator)	Setpoint within bounds, schedule changes, baseline archive
4	Significant action	Operator approval required	Equipment start/stop, large parameter change, twin retrain trigger
5	Critical action	Manager approval + dual auth	Plant-level shutdown, safety overrides, emergency response

7.5 Standard Workflows

Workflow A — Investigate Alert (autonomous)

Step	Agent	Action	Outcome
1	Trigger	Critical alert from FDD app	Workflow starts
2	Planner	Plan: fetch context, review diagnosis, check history, determine action	5-step plan
3	Executor	get_equipment, get_twin_diagnosis, get_fault_history, get_recent_work_orders	Context assembled
4	Validator	Confirm all data retrieved correctly. Synthesize findings.	Findings validated
5	Planner (re-plans)	Recommend create WO, notify technician, suggest parts	Action plan
6	Executor	create_work_order, send_notification, attach_parts_list	Actions executed
7	Validator	Confirm WO created, technician notified	Workflow complete

Workflow B — Step-Change Detected → Retrain Decision

Step	Agent	Action	Outcome
1	Trigger	step_change_detector ETL: error returned to baseline + 3-day persistence	STEP_CHANGE_DETECTED event
2	Planner	Determine retrain mode (auto/semi-auto). Plan: gather context, validate quality, decide	Plan based on config
3	Executor	get_recovery_period_quality, get_repair_wo_status, get_baseline_age, get_equipment_state	Full context
4	Validator	Verify: repair WO closed, recovery quality > 99% GOOD, sustained 3+ days, equipment NORMAL	Preconditions met
5a (auto)	Executor	transition_to_RETRAINING, archive_baseline, start_baseline_window	Retrain initiated

5b (semi-auto)	Executor	create_approval_request, notify_ai_ops_specialist with context, await	Pending approval
6	Executor (post-approval)	transition_to_RETRAINING, archive_baseline, start new baseline	Retrain proceeds
7	Validator	After retrain window: validate new baseline quality, confirm NORMAL transition	New baseline live

Workflow C — Daily Operations Report

Step	Agent	Action	Outcome
1	Trigger	Daily 6 AM scheduled trigger	Report generation begins
2	Planner	Plan: aggregate yesterday data, identify highlights, draft report, distribute	Report plan
3	Executor	Query telemetry, alerts, work orders, RL performance, twin diagnoses for prior 24h	Data assembled
4	Validator	Cross-check numbers vs DB. Flag any inconsistencies.	Numbers verified
5	Executor	Generate PDF, send to stakeholders	Report distributed
6	Validator	Confirm delivery. Log completion.	Done

7.6 Cost Model

Workflow Complexity	Cost per Workflow
Simple workflow (3-5 tool calls)	$0.05 – $0.15
Complex workflow (10-20 tool calls)	$0.20 – $0.50
Long-running workflow (50+ tool calls)	$1.00 – $3.00
Monthly cost per customer (typical, 50–500 sites)	$200 – $500
ROI	Saves 4–8 operator-hours per day per site = $400–$800/site/month

7.7 Cost Controls

- Per-workflow token budget — workflow stops if exceeds budget
- Monthly cost cap per customer — alerts at 80%, blocks at 100%
- Loop detection — Validator catches if Executor is repeating same tool calls
- Tool result caching — repeated identical queries served from cache (1-min TTL)
- Cost dashboard per customer per workflow type

PART 8 — Data Quality Strategy

8.1 Why ETL Is (Mostly) Sufficient

Graylinx's architecture absorbs most data quality issues through structural design choices, not through a heavy real-time quality service. This is a deliberate decision based on how each subsystem actually consumes data.

Structural Choices That Absorb Quality Issues
5-min poll + CoV — bounded latency; stale data is impossible (next poll catches up or marks offline)
Null-instead-of-zero for offline devices — distinguishes 'missing' from 'real zero' cleanly across all subsystems
Persistence in alerts (N consecutive readings) — handles spikes and brief noise automatically
Multi-point error patterns in FDD — single-sensor noise doesn't dominate fault classification
ETL'd data into FDD and twin training — pre-cleaned baseline, drift correction already applied
Mostly-binary RL inputs — robust to spikes, drift, noise, range issues by definition
Twin frozen during fault state — prevents contaminated retraining on degraded data

8.2 How Each Subsystem Gets Clean Data

Subsystem	Data Source	Why ETL Is Enough (or what extra is needed)
Twin (commissioning baseline)	ETL'd 2-week baseline data	ETL runs on schedule (15 min / hourly / daily). 2-week baseline window has plenty of time for cleaning before training. ✓ETL is sufficient.
Twin (FDD comparison)	ETL'd ongoing data	FDD app reads ETL'd data. Multi-point patterns + ETL = self-protecting. ✓ETL is sufficient.
RL Agent (binary inputs)	Real-time telemetry + small inline check	Binary signals robust to most issues. □ Need ~50 LOC frozen-binary detection in RL Broker.
RL Agent (building load)	Real-time telemetry	Continuous value but RL averages over time + equipment inertia. ✓Persistence absorbs occasional bad readings.
Alert Engine	Real-time telemetry + persistence + pre-checks	Persistence handles spikes/noise. □ Need 2 pre-checks: hard physical limits + frozen sensor detection.
LLM / Agentic AI	API queries + DWH (ETL'd)	Reads from API or DWH which is ETL-cleaned. ✓ETL is sufficient.
DWH Reporting	ETL'd data	ETL is purpose-built for this. ✓ETL is sufficient.

8.3 The Null-Handling Contract (Most Important)

This is a system-wide policy enforced from ingestion through every consumer. When a device is offline, Graylinx delivers null — never zero. Every subsystem is built to handle null correctly.

Layer	Null Handling Rule
Protocol Adapter	Device offline → emit null with quality_flag = 'MISSING'. NEVER coerce to 0, NEVER skip.
Transformer Module	Input null → output null. Do not apply scale/offset to null. Pass through unchanged.
Kafka Stream	Null values transit unchanged. quality_flag preserved.
TimescaleDB Storage	raw_value and transformed_value stored as NULL. quality_flag = 'MISSING'. NOT NULL constraints removed for value columns.
Alert Engine	Null comparisons return false (no rule fires on null). Track separately for offline detection. Fire DEVICE_OFFLINE after N consecutive nulls.
RL Broker	Null observation → apply fallback (last-known-good or safe default per config). NEVER pass null to agent (would crash or NaN-propagate).
Twin / FDD	Reads ETL'd data; ETL has already handled nulls per its rules (gap fill, exclusion, etc.).
ETL Pipeline	Aggregations skip nulls (never coerce to 0). Average of [22, 23, null, 24] = 23, not 17.25. Use SQL FILTER (WHERE value IS NOT NULL) explicitly.
API Endpoints	Return null in JSON for missing values. Document clearly. Frontend handles null gracefully (shows 'offline' badge).
UI Display	Null shown as 'OFFLINE' or '—' (not 0 or empty). Time-series charts: gap in line, not zero point.

8.4 Alert Engine Pre-Checks

Before evaluating equipment alert rules, the alert engine runs two cheap pre-checks. These run on every reading and add < 5ms latency.

Pre-Check 1: Hard Physical Limits

- Configured per point type (e.g., temperature: -50 to 100°C, pressure: 0 to 50 bar)
- Outside limits → SENSOR_FAULT alert (different category, routed to calibration team)
- Suppress equipment alerts on this point until sensor is fixed
- Distinguishes sensor problem from equipment problem

Pre-Check 2: Frozen Sensor Detection

- 5–10 sample window (configurable per point type)
- If standard deviation < threshold → SENSOR_FROZEN alert (different category, routed to calibration team)
- Suppress equipment alerts on frozen channel
- Resume normal alerting once sensor unfreezes

What's NOT in the alert engine

- Stale timestamp rejection — not needed because 5-min poll + CoV bounds latency
- Spike filtering — not needed because persistence requirement absorbs spikes
- Drift correction — handled in ETL and pushed back to transformer rules
- Range-vs-context checks — handled by FDD app on ETL'd data

8.5 RL Broker Inline Checks

The RL Broker has its own minimal data-handling logic — about 50 lines of code in total.

Check	Logic
Frozen Binary Detection	Per binary signal: track last N samples (default 60 minutes worth). If unchanged for entire window → flag frozen, apply fallback policy, log frozen_sensor_event.
Null Fallback	Per signal: if value is null, apply configured fallback. Default: last-known-good. Configurable: safe-default (0 for binary, last-known for continuous).
NaN Guard	Defensive: if any state vector element is NaN after all handling, do not call agent. Hold last action. Alert AI Ops Specialist.
Reward Quality Check	If reward signal sensor is BAD or MISSING, pause reward update for this step. Do not train on contaminated reward.

8.6 Device Offline Detection (Ingestion Layer)

Already part of the architecture — heartbeat monitor at the message queue layer.

- Each device has expected poll cadence (default 5 min)
- If no reading within poll_interval × 2 → mark device offline, deliver null to consumers
- Fire DEVICE_OFFLINE alert (separate category from equipment alerts)
- When device returns: deliver next reading normally, fire DEVICE_BACK_ONLINE info-level alert, log offline_event

8.7 ETL Quality Jobs (added to existing ETL)

These jobs run in the existing ETL framework (no new service). They produce sensor-level quality intelligence that feeds back into the runtime via transformer rules and twin calibration.

ETL Job	Purpose	Schedule	Output
sensor_drift_estimator	Compare each sensor to adjacent sensors over 7–30 day window. Fit drift model. Distinguish sensor drift from equipment degradation.	Daily	drift_coefficient → transformer_rules. sensor_health_score updated.
cross_sensor_validator	Check physical consistency across sensor pairs (inlet < outlet, supply < return, etc.). Flag persistent violations.	Hourly	Cross-sensor mismatch flags. Sensor mismatch alerts to calibration.
baseline_window_quality_scorer	During COMMISSIONING and RETRAINING states: monitor quality of incoming baseline data. Trigger window extension or quality alert if < 95% GOOD.	Hourly during commissioning	Window status. Quality score per device per window.
twin_calibration_feeder	Compute sensor correction offsets from drift estimation. Push updates to transformer rules.	Daily	Updated transformer rules. Twin sees corrected sensor baselines.
rl_experience_cleaner	Flag/remove imputed/bad observations from RL replay buffer.	Daily	Cleaned training dataset for RL retraining.
step_change_detector	Run on equipment in RECOVERING state.	Hourly during RECOVERING	Step-change events. Triggers

	Detect when error returns close to baseline error level. Require persistence (3+ days). Cross-check quality.		RETRAIN_PENDING transition.
quality_score_rollup	Aggregate per-point quality flags into device-level and site-level scores.	Hourly	device_quality_score, site_quality_score for dashboards.

8.8 Quality Issue Catalogue & Handling

For completeness, here is the full catalogue of issues and how each is handled by the right-sized strategy.

Sensor-Level Issues

Issue	Where Handled	Strategy
Sensor drift	ETL (sensor_drift_estimator)	Compute drift coefficient daily. Push to transformer rules. Apply correction at runtime.
Sensor bias	ETL (drift estimator detects offset)	Apply bias offset in transformer rules.
Frozen / stuck	Alert engine pre-check + RL broker	Detect via std-dev window. Fire SENSOR_FROZEN. Fallback in RL.
Spikes	Alert engine persistence	Persistence rule absorbs single spikes. FDD app on ETL'd data ignores spikes via multi-point patterns.
High-freq noise	ETL averaging + alert persistence	ETL produces aggregated values for FDD. Alert engine persistence absorbs.
Range / scale errors	Alert engine pre-check (hard limits)	Hard physical limits fire SENSOR_FAULT alert.
Calibration degradation	ETL + scheduled calibration WO	ETL flags. WO created. Manual recalibration.

System-Level Issues

Issue	Where Handled	Strategy
Missing data / gaps	Ingestion layer + null contract	Null delivered with MISSING flag. ETL gap-fills for DWH.
Late arrival	Not a problem (5-min poll + CoV bounds latency)	Bounded by poll interval. No special handling needed.
Duplicates	Ingestion layer	Dedup key (device_param_id, timestamp). Drop silently, log.
Inconsistent sampling	Ingestion + ETL	Adapter resamples to target. ETL produces regular time-series for analytics.
Protocol errors	Adapter layer	Map error codes to null with MISSING flag.
Timestamp issues	Adapter (UTC normalization)	Normalize to UTC. NTP sync at adapter.

Partial messages	Adapter (schema validation)	Reject. Affected points get null/MISSING.
Register misconfiguration	Commissioning tool + ETL cross-validation	Caught during commissioning. ETL flags ongoing inconsistencies.

Semantic Issues

Issue	Where Handled	Strategy
Physically impossible values	Alert engine pre-check (hard limits)	Hard physical limits fire SENSOR_FAULT, suppress equipment alerts.
Cross-sensor contradiction	ETL (cross_sensor_validator)	Hourly check. Persistent violations alert calibration team.
Context-impossible states	FDD app multi-point patterns	FDD detects inconsistent states across points and classifies fault.
Unit inconsistency	Commissioning tool + ETL distribution check	Caught during commissioning. ETL flags out-of-distribution values.
Rate-of-change impossibility	Alert engine pre-check (hard rate limit)	Configurable max delta/dt. Violation fires SENSOR_FAULT.
Seasonal / operating violation	FDD app + ETL baseline profiles	FDD compares to expected seasonal range, classifies as drift or fault.

8.9 Quality Dashboards

KPI	Target	Alert Threshold
Site quality score (rolling)	> 98%	Alert if < 95% for > 30 min
% Readings GOOD	> 99%	Alert if < 97% rolling 1-hour
% Readings MISSING (offline)	< 0.5%	Alert if > 2% per device
Sensors with active drift	0 (ideal)	Alert on first detection
Frozen sensor events (24h)	< 5 across portfolio	Alert if > 20
Devices offline > 1h	0 (ideal)	Alert per offline event
Sensor health score < 80	< 5% of fleet	Alert if > 10%
Twin baseline window quality (during commissioning)	> 99%	Alert if < 95% (extends window)

PART 9 — LLM Integration

9.1 Approach

LLMs power three things in Graylinx: (1) the Agentic AI multi-agent framework (covered in Part 7), (2) report generation, and (3) natural-language interfaces for technicians and operators. The same LLM provider strategy applies to all three.

9.2 Provider Strategy

Provider	When to Use	Trade-offs
Anthropic Claude (default)	Default for all use cases. Sonnet for most workflows, Opus for complex Planner agents.	Cost-effective, long context (200K), strong tool use, privacy-friendly (no training on API data).
OpenAI GPT-4 (alternative)	Customer contractually requires GPT-4. Need highest output quality.	Higher quality on some tasks, 3–5× cost vs Claude Sonnet.
On-prem Ollama (v3)	Customer requires data stays local — no LLM API calls allowed.	Lower quality, slower inference, requires GPU, but full data control.

9.3 Cost Model

Use Case	Typical Cost
Single report (batched, 10 reports/call)	$0.02–$0.05 per report
Single report (real-time)	$0.15–$0.25 per report
Simple agent workflow (3-5 tool calls)	$0.05–$0.15
Complex agent workflow (10-20 tool calls)	$0.20–$0.50
Long-running workflow (50+ tool calls)	$1.00–$3.00
Monthly per customer (typical mix)	$200–$500 (50–500 sites, batched reporting + Agentic AI workflows)

9.4 Cost Optimization Strategies

- Batched generation — combine 10+ reports in single LLM call (~80% cost reduction)
- Prompt caching — reuse system prompts across calls (Anthropic prompt caching feature)
- Progressive enrichment — start with minimal context, only add detail if response insufficient
- Tool result caching — repeated identical queries served from 1-min cache
- Model tiering — Sonnet for routine, Opus only for complex Planner agents
- Per-workflow token budgets — workflow stops if exceeds budget
- Per-customer monthly cost cap — alerts at 80%, blocks at 100%

9.5 Output Validation

Every LLM-generated output is validated before being shown to users or executed. Two layers of validation:

Layer	Description
Validator Agent	For Agentic AI workflows: dedicated Validator agent reviews Executor output, cross-checks against database, flags hallucinations.
Fact Validation	Numeric facts in reports cross-checked against source DB. Discrepancies > 1% flag the report for human review.
Schema Validation	Tool call inputs validated against tool schema before execution. Reject malformed.
Hallucination Tracking	Track validation failures over time per agent type. Target: < 2% hallucination rate.

9.6 Privacy & Data Redaction

Customer data sent to external LLM APIs (Claude, GPT-4) is redacted before transmission.

Redacted	Method
Site names	Replaced with anonymized IDs (SITE-1234)
Personnel names	Replaced with role-based pseudonyms (TECH-A, OPERATOR-B)
Email / phone	Removed from prompts
Customer name	Replaced with TENANT-X reference
Lat/lon coordinates	Generalized to region/timezone unless precision needed
Equipment serial numbers	Replaced with internal IDs unless specifically needed

On-prem Ollama (v3) eliminates this concern entirely — data never leaves customer infrastructure.

9.7 Rollout Phases

Phase	LLM Capabilities	Models Used
MVP (Month 3–6)	Batched executive reports, twin diagnostic reports, basic Agentic AI workflows (alert investigation, daily reports)	Claude Sonnet only
v2 (Month 6–9)	Custom agent workflows, expanded tool library, real-time agent activity, smart technician assignment, work order auto-description	Claude Sonnet + Opus for complex Planner agents
v3 (Month 10+)	Fully autonomous agents for low-risk operations, agent-to-agent collaboration, fine-tuned models, on-prem option	Claude (cloud) + Ollama (on-prem) options

PART 10 — HVAC Extension

10.1 Scope

The HVAC Extension adds specialized capabilities for heating, ventilation, and air conditioning systems. It extends Graylinx Core with HVAC-specific equipment types, fault codes, energy optimization, BMS integrations, and operator workflows.

10.2 HVAC Equipment Types

Equipment	Typical Parameters	Common Faults
Chillers (centrifugal, screw, scroll, air-cooled)	Supply/return temp, evap/cond pressure, compressor runtime, kW, COP, refrigerant level, valve position	Compressor failure, refrigerant leak, fouling, bearing wear
Air Handling Units (AHUs)	Supply/return air temp, fan RPM, fan kW, damper position, filter pressure, mixed air temp, OA fraction	Filter clog, belt slip, motor fault, damper stuck, freeze stat
Variable Air Volume (VAV)	Zone temp, setpoint, damper position, airflow, reheat valve	Damper stuck, reheat valve stuck, sensor fault
Fan Coil Units (FCU)	Zone temp, fan speed, valve position, mode (heat/cool)	Coil leak, fan motor, valve stuck
Boilers	Supply/return water temp, pressure, flame status, gas flow, efficiency, stack temp	Burner fault, flame failure, scaling, low water cut-off
Heat Exchangers	Inlet/outlet temps both sides, flow rates, pressure drop, effectiveness	Fouling, leak, blocked tubes
Pumps	Speed, flow, head, kW, vibration	Cavitation, bearing wear, seal leak, motor fault
Cooling Towers	Inlet/outlet water temp, fan speed, water level, conductivity, flow	Drift, scaling, fan motor, fill fouling
Thermostats / Sensors	Temperature, humidity, CO2, occupancy	Calibration drift, dead sensor, mounting issue

10.3 HVAC-Specific Modules

Module 2A — HVAC Alerting & Fault Codes

Feature	Description
HVAC Fault Code Library	100+ fault codes mapped from BACnet/Modbus/proprietary protocols to human-readable descriptions, root causes, recommended actions, parts lists.
Temperature Threshold Alerts	Supply air, return air, zone, outdoor air. Configurable per equipment.
Pressure Alerts	Discharge, suction, differential. High/low limits.
Flow Rate Alerts	GPM, CFM out of range. Indicates blockage, fan failure, pump issues.
Compressor Runtime Alerts	Excessive runtime (>18h/day) indicates oversizing or setpoint issues.
Recommended Actions Per Fault	'High discharge temp → Check for overcharging, condenser fouling. See manual section X.'
Schedule Breach Alerts	Equipment running during unoccupied hours.
Setpoint Deviation Alerts	Setpoint changed but zone not meeting target.

Module 3A — Energy Optimization

Feature	Description
Energy Consumption Dashboard	kWh per site, per chiller, per AHU. Cost in $/month. Comparison to baseline + last month.
Equipment Power Trending	Chiller kW, compressor kW, fan kW. Identify high consumers.
RL-Driven Setpoint Optimization	User sets comfort band. RL agent learns optimal setpoint within band for energy savings + comfort.
Occupancy-Based Scheduling	Different setpoints for occupied vs unoccupied. Auto-applied per occupancy calendar.
Demand Response	Utility peak signal → relax setpoints temporarily. Estimate kWh saved.
Energy Anomaly Reports	Unusual consumption patterns. LLM-recommended causes (oversizing, fouling, schedule drift).
Multi-Site Energy Comparison	Rank sites by kWh/sqft. Identify outliers for investigation.
Carbon Tracking (v3)	Compute CO2e from kWh × regional grid mix. Track vs targets.
Demand Forecasting (v3)	Predict next-week demand from weather + occupancy forecast.

Module 4A — HVAC Controls & Write-Back

Feature	Description
Setpoint Adjustment	Remote setpoint change via BACnet/Modbus write. Audit logged.
Mode Control	Heat/cool/auto/off per zone or equipment.
Fan Speed Control	Off/low/medium/high or 0–100%. VAV damper positions.
On/Off Toggle	Start/stop equipment. Confirmation dialog + supervisor approval (Tier 4).
Occupancy Override	Manual occupied/unoccupied. Override duration (15min, 1h, 4h, until next schedule).
Demand Response Commands	Auto-adjust setpoints during peak events. Operator can override.
Audit Trail	Every write logged: who, when, what, old value, new value, reason. Compliance-ready.

Module 5A — HVAC Predictive Maintenance

Feature	Description
Twin-Driven PM	Twin RUL estimates inform PM timing. PM when twin says RUL approaching threshold, not on fixed calendar.
HVAC PM Task Library	Filter replacement (90d), compressor oil change (annual), condenser cleaning (quarterly), refrigerant top-up.
Auto-WO Generation	Twin diagnosis or PM schedule auto-creates WO. Pre-populated with parts, instructions, technician skill match.
Warranty Tracking	Track equipment warranty expiry. Alert before expiration.
Spare Parts Inventory	Recommended parts per equipment type. Alert when low stock.
PM Compliance Reporting	% scheduled PM completed on time. Overdue items flagged.
Failure Risk Scoring	Per-equipment risk score (0–100) updated daily from twin error trend + RL performance.

10.4 HVAC Integrations

BMS Integration

BMS Vendor	Integration Approach
Siemens Desigo CC	REST API for read; OPC-UA bridge for write-back. Phase 1 read-only, Phase 2 bi-directional.
Honeywell EBI	Honeywell APIs + native BACnet integration. Phase 2.
Johnson Controls Metasys	Metasys API + BACnet/IP. Phase 2.
Schneider EcoStruxure	EcoStruxure REST API + BACnet. Phase 2.
Trane Tracer	Native BACnet (most exposed). Phase 1.
Generic / unknown BMS	BACnet/IP discovery + Modbus TCP. Manual point mapping in Commissioning Tool.

CMMS Integration

- IBM Maximo — bi-directional WO sync
- ServiceNow — bi-directional WO sync via REST API
- UpKeep — webhook-based integration
- Generic — REST API + webhook framework for custom CMMS

Energy Management

- OpenADR for demand response signaling
- Utility tariff integration for cost-aware optimization
- Weather data (NOAA, Met Office) for forecasting

10.5 HVAC-Specific Personas

Energy Manager

Monitors energy consumption across portfolio. Drives RL optimization configuration and tuning. Reports to executive on savings.

- Tools: desktop dashboards, energy reports, export
- Goals: reduce energy 10–25%, meet sustainability targets

Senior HVAC Technician

Handles complex faults that junior staff escalate. Supervises maintenance teams. Reviews twin diagnoses and validates AI-recommended actions.

- Tools: tablet in field, manuals, parts catalog
- Goals: minimize MTTR, prevent repeat failures, mentor juniors

Facilities Manager

Owns operational KPIs for one or more facilities. Reviews twin lifecycle dashboards, approves twin retrains in semi-auto mode, oversees PM compliance.

- Tools: desktop, mobile dashboard
- Goals: uptime, comfort, energy efficiency, technician productivity

10.6 HVAC Reports

- Daily — Site health summary, alerts, energy
- Weekly — Equipment performance trending, WO completion, technician productivity
- Monthly — Energy analysis with optimization recommendations, PM compliance, twin health
- Quarterly — Lifetime equipment degradation, ROI on repairs, replacement recommendations
- Annual — Sustainability report, carbon, fleet age, capital planning input

PART 11 — Factory Extension (Slim)

11.1 Scope

The Factory Extension adds capabilities for industrial manufacturing operations. The scope is intentionally focused: monitoring, OEE, and maintenance only. Production scheduling, MES integration, and quality/defect management are explicitly out of scope — Graylinx integrates with the customer's existing systems for those concerns.

What's IN Scope
Production line and machine monitoring (real-time visibility)
OEE calculation and tracking (Availability × Performance × Quality)
Maintenance management (predictive + preventive + reactive)
Twin-driven fault detection on factory equipment
RL optimization within safety bounds
Agentic AI workflows for routine factory operations

What's OUT of Scope
Production scheduling — handled by customer's MES/ERP
MES integration as primary system — Graylinx is observer/sidecar, not replacement
Quality / defect management — handled by customer's quality system
Recipe management for process manufacturing — stays in customer's recipe system
Compliance / batch records — Graylinx feeds data to customer's compliance system

11.2 Manufacturing Types Supported

The platform supports both discrete and process manufacturing. Specific applications and equipment types will be decided per asset and per industry.

Type	Examples	Characteristics
Discrete Manufacturing	Assembly lines, machining, electronics, automotive parts, consumer goods	Discrete units. Cycle time per unit. Clear start/stop boundaries. OEE based on units produced.
Process Manufacturing	Chemical, food & beverage, pharma, oil & gas refining, water treatment	Continuous flow. Throughput per hour. Batch or continuous mode. OEE based on production volume.
Hybrid / Batch	Pharmaceuticals, specialty chemicals, food processing	Batch-based discrete units. Mix of process and discrete characteristics.

Specific equipment types, sensor configurations, and applications are determined during commissioning per customer and per industry. The platform is intentionally generic at the core, with industry/asset specifics defined via the Data Access Model.

11.3 Module 1F — Production Monitoring

Feature	Description
Line Status Dashboard	Real-time status of every line: running, idle, stopped, faulted. Per-line throughput. Bottleneck indicator.
Machine Status Dashboard	Per-machine status, current cycle/batch, runtime, downtime. Twin lifecycle state visible.
Real-Time Telemetry	Process variables (temp, pressure, flow, RPM, vibration). Live trends. Alert overlay.
Production Counters	Discrete: units produced, units rejected. Process: throughput rate, cumulative volume.
Downtime Tracking	Auto-detect downtime events. Categorize: planned / unplanned / changeover / setup / breakdown.
Downtime Reasons	Operator can attribute downtime to a reason from configurable library. Auto-suggest from telemetry pattern.
Shift View	Per-shift KPIs (production, downtime, OEE). Compare shifts.
Equipment History	Drill-down: recent runs, runtime, downtime events, alarms, work orders, twin diagnoses.

11.4 Module 2F — OEE (Overall Equipment Effectiveness)

OEE is the standard manufacturing metric: Availability × Performance × Quality. Graylinx computes OEE in real-time and tracks it over time.

Component	Formula	What Reduces It
Availability	Run Time / Planned Production Time	Unplanned downtime, breakdowns, changeover delays
Performance	(Ideal Cycle Time × Total Count) / Run Time	Slow cycles, minor stops, throughput below standard
Quality	Good Count / Total Count	Defects, rework, scrap (note: defect data must come from customer's quality system, fed in via integration)
OEE	Availability × Performance × Quality	World-class is 85%+, typical is 60%, opportunity if < 50%

OEE Features

Feature	Description
Real-Time OEE	Live OEE per machine, per line, per shift. Updates as runtime accumulates.
OEE Trending	Daily / weekly / monthly trends. Identify improvement or degradation.
OEE Components Drill-Down	When OEE drops, identify which component (A, P, Q) is the cause. Specific event log.
Loss Categorization	Map each loss to one of the Six Big Losses: breakdowns, setup/changeover, idle/minor stops, reduced speed, startup defects, production defects.
Comparative OEE	Compare lines, shifts, products, sites. Identify benchmark and improvement targets.
OEE Goals & Tracking	Set OEE targets per line. Track progress. Alert if trending wrong way.
Twin-Driven OEE Insights	Twin diagnoses contribute to OEE explanation: 'Performance dropped 8% this week — twin shows compressor inefficiency on Line 3.'
OEE Reports	Auto-generated OEE reports (daily / weekly / monthly). LLM-summarized insights.

11.5 Module 3F — Maintenance Management

Feature	Description
Predictive Maintenance (Twin-Driven)	Twin RUL estimates trigger maintenance before failure. PM scheduled when RUL crosses threshold, not on calendar.
Preventive Maintenance Library	Standard PM tasks per equipment type: lubrication, calibration, inspection, replacement intervals.
Reactive Maintenance	Standard repair WOs from breakdown alerts. Twin diagnosis pre-populates fault and recommended fix.
Maintenance Calendar	Visual calendar of all scheduled and pending maintenance. Conflict detection.
Technician Dispatch	Auto-assign based on skills, location, current load. Agentic AI suggests best technician.
MTBF / MTTR Tracking	Mean Time Between Failures, Mean Time To Repair per machine and per fault type.
Spare Parts Catalog Link	Each PM task and fault diagnosis links to required parts. Inventory check before WO dispatch.
PM Compliance Tracking	% scheduled PM completed on time. Overdue PM flagged.
Cost Tracking	Labor + parts per WO. Rolled up per machine, per category. Identify cost drivers.
Failure Mode Analysis	Pareto of failure modes per equipment. Identify chronic issues for redesign or replacement.

11.6 Factory-Specific Considerations

Twin Lifecycle Management for Factory Equipment

- Commissioning baseline window: 2 weeks (default), but may extend for batch/seasonal equipment that runs in distinct modes
- For process equipment: baseline must capture full production cycle including startup, steady state, shutdown
- For discrete equipment: baseline captures multiple production runs of typical product mix
- Retrain after major maintenance (gearbox replacement, motor swap, PLC firmware update)

RL for Factory Operations

- Throughput optimization within quality constraints
- Setpoint optimization for process variables
- Energy minimization while maintaining production
- Buffer management across line stages
- Constraint: must stay within recipe/specification bounds (recipe is owned by customer's MES, Graylinx respects it)

Agentic AI for Factory Operations

- Auto-investigate alerts during off-shifts
- Pre-populate maintenance WOs with diagnosis
- Generate shift handover reports
- Identify bottlenecks across the line
- Coordinate maintenance windows with production schedule (read-only — production schedule owned by MES)

11.7 Factory Integrations

System	Integration Type	Direction
MES (Wonderware, Rockwell FT, SAP ME, Siemens MOM)	REST API + OPC-UA where supported	Read production schedule, batch info. Write nothing.
ERP (SAP, Oracle, Microsoft Dynamics)	REST API or middleware bridge	Read inventory levels, write WO completion data.
Quality System	REST API or file drop	Read defect counts for OEE Quality calculation.
PLC / SCADA	OPC-UA, Modbus TCP, Profinet, EtherNet/IP	Read telemetry. Write controls (with safety bounds).
CMMS	REST API + webhooks	Bi-directional WO sync.
Energy Meters	Modbus TCP, BACnet/IP	Read kWh, kVA, peak demand.

11.8 Factory-Specific Personas

Production Manager

Responsible for shift output and OEE. Monitors live production, investigates downtime, drives improvement initiatives.

- Tools: large-screen dashboard, mobile alerts
- Goals: OEE targets, throughput, on-time delivery

Maintenance Lead

Manages maintenance team and PM schedules. Prioritizes repairs, balances workloads, ensures spare parts available.

- Tools: desktop dashboard, tablet for floor walks
- Goals: minimize unplanned downtime, PM compliance, cost control

Operator

On the floor, tends one or more machines. Acknowledges alarms, performs minor adjustments, escalates issues, attributes downtime reasons.

- Tools: machine HMI, tablet, kiosk
- Goals: keep machine running, meet shift quota, accurate downtime logging

Reliability Engineer

Analyzes failure patterns, configures twin baselines, tunes PM intervals, identifies chronic issues for redesign.

- Tools: desktop analytics, twin diagnostic dashboards, historical reports
- Goals: MTBF improvement, root-cause elimination, capital planning

PART 12 — Roadmap

12.1 Phased Delivery Plan (24 months)

Phase	Timeline	Theme	Outcome
Phase 0	M1 – M2	Foundation	Architecture validated, tech stack stood up, beta customer signed
Phase 1	M3 – M6	MVP	Production-ready core platform, HVAC extension, beta deployment
Phase 2	M7 – M9	v2 (Twin lifecycle + Agentic AI)	Full twin lifecycle, Agentic AI workflows, factory extension
Phase 3	M10 – M24	v3 (Maturity & Scale)	Production-grade autonomy, on-prem LLM option, multi-vertical depth

12.2 Phase 0 — Foundation (M1–M2)

Goals

- Validate core architecture decisions with proof-of-concept
- Sign first beta customer with signed scoping document
- Stand up development infrastructure

Deliverables

- Reference architecture document (this document)
- Working POC: end-to-end data flow from one BACnet device to dashboard
- Database schema v1, migrations framework
- Kafka topic design, partitioning strategy
- Authentication (Keycloak) integrated
- Beta customer signed with 5–10 site pilot scope
- CI/CD pipeline, dev/staging/prod environments
- Observability stack (Prometheus, Grafana, Loki) deployed

12.3 Phase 1 — MVP (M3–M6)

Goals

- Production-ready Graylinx core platform
- HVAC extension complete
- Beta deployed to 5–10 sites
- Customer UAT signed off

Core Platform Deliverables

Module	MVP Scope
Real-Time Monitoring	Site dashboard, device detail, live telemetry, parameter write-back, multi-device compare
Alerting	Real-time threshold + persistence rules. Hard physical limits + frozen detection pre-checks. Notification delivery.
Reporting	Daily, weekly, monthly templates. Custom export. Basic LLM-generated narrative summaries.
Work Orders	Full lifecycle. Mobile UI. Agentic AI auto-creates from alerts. CMMS integration (1 vendor).
Scheduling	Schedule builder, automated execution, conflict detection.
Portfolio Management	Multi-site hierarchy, RBAC, audit logging, multi-tenancy.
Integration	BACnet, Modbus, OPC-UA, MQTT, REST adapters. REST API + GraphQL. Initial CMMS sync.
Commissioning Tool	Source/device/point/mapping configuration. Bulk import. Live config push.
Twin & FDD	Manual baseline capture. NORMAL + FAULT lifecycle states. FDD app on ETL'd data. Multi-point pattern detection.
RL	RL Broker. Shadow + production modes. Frozen-binary detection. Twin lifecycle pause integration.
Agentic AI	Multi-agent framework. Planner / Executor / Validator. 3 standard workflows. Tool registry v1.
Data Quality	Null contract enforced everywhere. Alert pre-checks live. ETL drift estimation + cross-sensor validation. Quality dashboards.
LLM	Claude Sonnet integration. Batched report generation. Validator agent for hallucination defense.

HVAC Extension Deliverables

- HVAC equipment types defined (chillers, AHUs, VAVs, boilers, heat exchangers, pumps)
- HVAC fault code library (initial 50 codes)
- Energy consumption dashboard

- BMS integration: BACnet/IP + 1 vendor (Trane Tracer or generic Modbus BMS)
- HVAC-specific alert rules and reports
- Setpoint optimization (rule-based; RL in Phase 2)

12.4 Phase 2 — v2 (M7–M9)

Goals

- Full twin lifecycle (all 9 states)
- Agentic AI workflow library expansion
- Factory extension delivered
- Customer count: 3–5

Twin Lifecycle Enhancements

- All lifecycle states implemented (COMMISSIONING, TRAINING, NORMAL, DEGRADING, FAULT, RECOVERING, RETRAIN_PENDING, RETRAINING, RETIRED)
- Versioned baseline storage and archival
- Step-change detection ETL job
- Retrain orchestrator (auto and semi-auto modes)
- Lifetime degradation tracking and reporting
- Twin lifecycle dashboards
- Configurable retrain mode per equipment

Agentic AI Enhancements

- Tool library expansion (additional 20+ tools)
- Workflow library: 10+ standard workflows
- Real-time agent activity feed
- Smart technician assignment
- Approval queue UI for AI Ops Specialist
- Cost tracking per workflow / per customer
- Loop detection and cost guardrails

Factory Extension Deliverables

- Production line monitoring
- OEE calculation and dashboards
- Maintenance management (predictive + preventive + reactive)
- MES read-only integration (1 vendor)
- PLC/SCADA via OPC-UA, Modbus, EtherNet/IP
- Factory-specific personas in UI
- Both discrete and process manufacturing support

HVAC Enhancements

- Additional BMS vendors (Siemens, Honeywell, JCI)

- RL-driven setpoint optimization
- Demand response (OpenADR)
- Carbon tracking
- Expanded fault code library (200+ codes)

12.5 Phase 3 — v3 (M10–M24)

Goals

- Production-grade autonomy (60%+ workflows fully autonomous)
- On-prem LLM option (Ollama)
- Vertical depth (more BMS/MES integrations, deeper fault libraries)
- Customer count: 10–20

v3 Capabilities

Capability	Description
Full Agent Autonomy	60%+ of routine workflows execute end-to-end without human intervention. Approval only for high-risk actions.
On-Prem LLM (Ollama)	Customers requiring strict data residency can run LLM locally on GPU node. Lower quality but full data control.
Fine-Tuned Models	Custom model fine-tuning per customer or per vertical for specialized vocabulary and workflows.
Predictive Sensor Failure	Predict sensor failure before it happens (drift trajectory + age + environmental).
Cross-Site Pattern Detection	Identify patterns across the entire portfolio (e.g., chiller model X has 30% higher fault rate after year 5).
Auto-Calibration Recommendations	Agentic AI proactively recommends calibration based on drift estimates and sensor age.
RL for Scheduling	RL agents learn optimal scheduling: when to run/pause equipment, batch sizing, sequence optimization.
Multi-Site RL	RL agents that optimize across sites (energy, technician dispatch, parts inventory).
Mobile App	Native iOS/Android apps for Operations Manager and Technician personas.
Voice Interface	Natural-language voice commands for hands-free operation in noisy environments.
Additional Verticals	Water treatment, power distribution, oil & gas extensions.

12.6 Success Metrics by Phase

MVP Metrics (Month 6)

Metric	Target
Sites onboarded	5–10 (1 beta customer)
Pipeline uptime	≥ 95%
Real-time UI latency	< 5 seconds
Twin FDD coverage	Detect 80%+ faults ≥24h before failure
RL improvement	10%+ on tracked KPI within 30 days
Agent autonomy	60%+ routine workflows autonomous, < 5% incorrect actions
Customer UAT	Signed off by Month 6

v2 Metrics (Month 9)

Metric	Target
Customers	3–5
Sites total	50–100
Twin lifecycle states all live	9/9 states implemented
Step-change detection accuracy	> 90%
Agentic AI workflows	10+ standard, 5+ custom per customer
Factory extension live	1+ factory customer

v3 Metrics (Month 24)

Metric	Target
Customers	10–20
Sites total	500–2,000
Verticals supported	HVAC + Factory + 2 more
On-prem LLM customers	1+ deployed
Agent autonomy rate	60%+ across all customers
Cumulative energy savings (HVAC)	$1M+ across customers
Cumulative downtime avoided	10,000+ hours

12.7 Risks & Mitigations

Risk	Impact	Mitigation
Customer-built twin models incompatible with FDD app	FDD can't classify faults	Define standardized twin output schema in Phase 0. Validation harness for twin onboarding.
BMS vendor APIs change	Integration breaks	Adapter pattern with versioning. Integration tests run on schedule. Vendor relationships.
LLM cost overrun	Margin erosion	Per-workflow budget. Per-customer cap. Batched generation. Prompt caching.
Hallucinated agent actions	Wrong actions in field	Validator agent. Approval gates. Tier-based authorization. Audit trail.
Customer requires features not in roadmap	Scope creep, delays	Strict change control. Vertical extensions absorb most asks. Roadmap conversation upfront.
Data quality bad enough to break twin baseline	Bad twin → bad FDD	Commissioning quality scoring. Window extension. AI Ops Specialist intervention.
Unplanned RL action causes outage	Service disruption, customer trust	Hard safety bounds. Shadow mode default for new agents. Twin pause integration.
On-prem deployment complexity	Customer install delays	Helm charts, automated install scripts, professional services for first 5 customers.
Multi-tenant data leakage	Customer breach, regulatory	Row-level security in DB. JWT tenant_id enforced at API. Penetration testing.

PART 13 — Appendix

A.1 Glossary

Term	Definition
Agentic AI	Multi-agent AI system where specialized agents (Planner, Executor, Validator) collaborate to execute multi-step workflows with tool use.
AI Ops Specialist	New role created by Graylinx. Configures and tunes Agentic AI agents, RL reward functions, and twin baselines. Reviews agent decisions and approves twin retrains.
BACnet	Building Automation and Control Networks protocol. Industry standard for HVAC and building systems.
Baseline (Twin)	Twin model trained on a specific data window. Each piece of equipment has a versioned history of baselines.
BMS	Building Management System. The umbrella software for HVAC + lighting + access control in a building.
CMMS	Computerized Maintenance Management System. Used to track work orders, parts, and maintenance schedules.
Commissioning Window	First 2 weeks of post-installation operation, used to capture the twin baseline.
CoV	Change of Value. Polling strategy that emits a reading only when value changes (not on fixed cadence).
DAM	Data Access Model. Graylinx's structured definition of which physical inputs map to which logical points, with all metadata.
DDC	Direct Digital Controller. A controller that directly interfaces with field devices.
DWH	Data Warehouse. ETL target for historical analytics and reporting.
FDD	Fault Detection and Diagnosis. Independent application that compares twin predictions to actual operation and classifies faults.
Hypertable	TimescaleDB time-partitioned table. Automatically chunks data by time for fast queries and compression.
LKG	Last Known Good. A fallback strategy where the most recent valid value is reused when current value is missing or bad.
MES	Manufacturing Execution System. Manages production scheduling, work-in-progress, batch records on the factory floor.
Multi-Point Pattern (FDD)	Fault detection that evaluates error vectors across multiple equipment points simultaneously, looking for known fault signatures.
Null Contract	System-wide policy that null means 'device offline / data missing' and is treated consistently by every subsystem. NEVER coerced to 0.

OEE	Overall Equipment Effectiveness. Manufacturing metric: Availability × Performance × Quality.
OPC-UA	OPC Unified Architecture. Industrial protocol for machine-to-machine communication.
PM	Preventive Maintenance. Scheduled (calendar or runtime-based) maintenance.
PLC	Programmable Logic Controller. Industrial controller for automation.
Persistence (Alerts)	Requirement that a threshold breach must hold for N consecutive readings before firing an alert. Eliminates spike-based false positives.
RL	Reinforcement Learning. Agent learns to maximize reward through interaction with environment.
RUL	Remaining Useful Life. Twin-estimated time before equipment failure.
Step-Change Detection	FDD detection of error returning toward baseline level after repair, indicating recovery and triggering retrain decision.
Tier (Approval)	Approval level for an Agentic AI action. Tier 1 (read-only autonomous) through Tier 5 (critical, dual auth).
Twin Lifecycle State	One of 9 states (COMMISSIONING / TRAINING / NORMAL / DEGRADING / FAULT / RECOVERING / RETRAIN_PENDING / RETRAINING / RETIRED) tracking the twin's relationship to the equipment.
VAV	Variable Air Volume terminal unit. HVAC component delivering conditioned air to a zone.
Validator (Agent)	Agentic AI role that verifies Executor output and cross-checks against database. Hallucination defense.
Write-Back	Sending a command from Graylinx to a device (setpoint change, mode change, on/off).

A.2 Design Decisions Log

Critical architectural decisions made during design, with rationale and alternatives considered.

Decision	Choice	Alternatives	Rationale
Twin Approach	Commissioning baseline + frozen during fault, with versioned baselines	Continuous prediction with rolling baseline; physics-only models	Captures cleanest reference (as-installed). Frozen during fault preserves diagnostic signal. Versioning enables lifetime ROI analysis.
FDD Architecture	Independent app on ETL'd data, multi-point patterns	Integrated into twin runtime, single-point error detection	FDD can evolve independently. Multi-point patterns naturally noise-resistant. ETL'd data is pre-cleaned.
Data Quality Strategy	ETL + null contract + alert pre-checks (right-sized)	Full Tier-1 real-time quality service	Architecture absorbs most quality issues structurally (5-min poll, multi-point FDD, null-instead-of-zero, persistence). Heavy quality service would be redundant.
RL State Observation	Mostly binary + real-time building load	Continuous values throughout; full state vector	Binary signals robust to most quality issues by design. Reduces quality-layer requirements substantially.
Null Handling	Null delivered for offline (never zero); system-wide contract	Coerce to 0; skip silently	Distinguishes 'missing' from 'real zero' cleanly. Prevents silent corruption of aggregations and decisions.
Agentic AI Architecture	Multi-agent (Planner / Executor / Validator)	Single LLM with tool use	Validator catches hallucinations. Separation enables audit. Cost control easier.
LLM Provider	Anthropic Claude (default), OpenAI alternative, Ollama for on-prem	Single-vendor lock-in	Cost-effective, long context, strong tool use, on-prem path for data-residency customers.
Polling Strategy	5-min poll + CoV	Streaming-only (CoV); fixed-rate poll	Heartbeat guarantees liveness. CoV reduces volume 10x. Bounds latency for stale data.
Deployment Default	On-premise	Cloud-first SaaS	Industrial customers want data sovereignty. Cloud is option, not default.

Streaming Backbone	Kafka	Pulsar; RabbitMQ; in-process	Mature, durable, partitioned fan-out. Replay capability essential.
Time-Series DB	TimescaleDB	InfluxDB; ClickHouse	PostgreSQL extension — same DB family as transactional data. SQL-compatible. Mature.
Retrain Trigger	Configurable: auto OR semi-auto (engineer approves)	Always auto; always manual	Customer choice based on risk tolerance. Different equipment classes can have different policies.
Baseline Retention	Keep all baselines forever (versioned)	Keep last N; delete after retrain	Enables lifetime ROI analysis. Storage cost negligible vs analytical value.
Factory Scope	Monitoring + OEE + maintenance only	Full MES replacement	Customer's MES/ERP/quality systems are mature. Graylinx integrates, not replaces.

A.3 Open Questions

Issues to resolve during Phase 0 / Phase 1 implementation.

Question	Plan to Resolve
Twin module schema standardization	Phase 0: define standardized input/output schema for customer-built twin modules. Validation harness for twin onboarding.
RL agent runtime isolation	Phase 0: containerize each customer's RL agents in isolated runtimes. Validate resource limits and security boundaries.
FDD multi-point pattern library	Phase 1: build initial library of 20–30 known fault patterns per equipment type. Iteratively extend based on real customer faults.
Step-change detection thresholds	Phase 2: tune thresholds per equipment type using historical data from beta customers. Default + per-customer override.
Cross-vendor BMS integration depth	Phase 1–2: prioritize 1 vendor for MVP, expand based on customer demand. Agentic AI to assist in onboarding new BMSes.
Factory equipment-specific applications	Decided per asset and per industry during commissioning. Platform stays generic; vertical knowledge captured in DAM and fault libraries.
Cost model for very large customers (1000+ sites)	Phase 3: validate LLM and infrastructure cost scaling. Volume discounts. On-prem LLM may be required for biggest customers.
Compliance certification roadmap	SOC 2 (Phase 1), ISO 27001 (Phase 2), HIPAA / FDA 21 CFR Part 11 (Phase 3 if pharma customer signed).
Mobile app priorities	Phase 3: tablet web UI works for MVP. Native iOS/Android based on customer demand.
International expansion	Phase 3+: data sovereignty per region (EU, APAC). Localization. Regional support.

A.4 Document Changelog

Version	Date	Changes
v1.1.0	May 2026	Master consolidated document. Combines: PRD, architecture, data model, modules, twin lifecycle, RL, Agentic AI, data quality (right-sized), LLM, HVAC extension, factory extension (slim), roadmap, appendix.
v1.0	Apr 2026	Initial Graylinx PRD (HVAC-only).

End of Document
Graylinx Industrial Platform — Master Product & Architecture Specification
v1.1.0 (Phase 1 Draft) — May 2026
Total: 13 parts spanning PRD, architecture, twin lifecycle, RL, Agentic AI, data quality, LLM integration, HVAC + factory extensions, and 24-month roadmap